

ИТМО	Министерство науки и высшего образования Российской Федерации
	федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО» (Университет ИТМО)
	Правила проведения конкурса «Лучшие научные работы стажировки НОЛ ТЗ» ПКУ-СМК-_____ страница 10 из 12

Приложение 2

к правилам проведения конкурса

«Лучшие научные работы стажировки НОЛ ТЗ»

ОТЧЕТ

ПО НАУЧНОЙ РАБОТЕ СТАЖИРОВКИ НОЛ ТЗ

Тема:	Идентификация паттернов поведения лабораторных крыс по последовательности видеок кадров
ФИО стажёра:	Ветров Максим Алексеевич
ФИО руководителя:	Волынский Максим Александрович

Отчет на 31 листе

Подпись стажёра:



УДК 004.825

М. А. Ветров, бакалавр, e-mail: maxvetrov2004@yandex.ru,

Уральский федеральный университет имени первого Президента России
Б.Н. Ельцина

Идентификация паттернов поведения лабораторных крыс по последовательности видеок кадров

Ключевые слова: *идентификация паттернов поведения, лабораторные крысы, анализ видеок кадров, ключевые точки, ограничивающий прямоугольник, фильтр Калмана, классификация поведения, Random Forest, медианный фильтр, усреднение результатов, автоматизация мониторинга.*

Введение

Целью данного исследования является разработка и тестирование методов идентификации паттернов поведения лабораторных крыс на основе анализа видеок кадров. Для достижения этой цели необходимо выполнить следующие задачи: сбор и подготовка данных, разработка алгоритмов классификации, а также их анализ и оптимизация. На этапе сбора данных требуется провести разметку видеок кадров для выделения координат *ограничивающего прямоугольника* [1], который представляет собой область, очерчивающую границы объекта и указывающую его местоположение и размер. Также необходимо определить координаты *ключевых точек* [2], которые являются стратегически важными областями на теле крысы, такими как суставы и конечности. Анализ этих точек позволяет выявить характерные паттерны поведения [1], повторяющиеся движения или специфические позы. Таким образом, использование ключевых точек для классификации паттернов поведения [1] может автоматизировать мониторинг состояния объектов.

Постановка задачи и алгоритмы, рассматриваемые в статье, были разработаны в рамках стажировки в Научно-образовательной лаборатории «Техническое зрение» университета ИТМО. Результаты исследования в дальнейшем будут использоваться для работы над выявлением паттернов

поведения пациентов в больнице, что позволит в реальном времени, без участия врачей, наблюдать за пациентами.

1 Подготовка данных

На этапе подготовки была выполнена разметка данных [2]. Разметка данных представляет собой процесс аннотирования кадров на видео с целью выделения ключевых объектов или областей интереса. Этот процесс является критически важным для подготовки данных к исследованию, так как качественно размеченные данные позволяют значительно повысить точность и надежность алгоритмов компьютерного зрения. Система использует одну видеокамеру, установленную в верхней части террариума. Камера снимает в секунду 30 кадров. Каждый видеокادر имеет размер 1280x720 пикселей.

Существует два основных подхода к разметке данных: автоматизированный и ручной. Автоматическая разметка использует алгоритмы для выделения объектов на изображениях или видео, что позволяет обрабатывать большие объёмы данных за короткое время. Однако такой подход не всегда обеспечивает такие же качественные результаты, как ручная разметка.

Ручная разметка, выполняемая человеком, позволяет достичь высокой точности и качества аннотаций. В рамках исследования разметка части данных была выполнена вручную с использованием инструмента CVAT (Computer Vision Annotation Tool) [4], который был запущен из Docker-контейнера на операционной системе Linux. CVAT – это открытая платформа для аннотирования изображений и видео, предоставляющая широкий набор инструментов для создания точных разметок. С помощью CVAT было размечено видео, состоящее из 506 кадров.

Ручная разметка позволяет получить более точные данные по сравнению с автоматической, однако она требует значительных временных затрат и участия человека. Это делает её менее подходящей для разметки больших объёмов данных и систем мониторинга в реальном времени. В связи с этим остальные данные, использованные для тестирования алгоритма, были

размечены автоматически. Это позволило сократить время, затрачиваемое на разметку, и адаптировать алгоритм к реальным данным.

Для удобства размеченные данные были разделены на наборы данных по категориям: ходит (*rearing*), сидит (*sitting*), встаёт на задние лапы (*rearing*), умывается (*grooming*). Тестирование алгоритмов на различных наборах данных позволило более эффективно адаптировать классификатор к различным паттернам поведения крыс.

2 Постановка задачи

Введем следующие понятия и определения.

Назовем *ключевыми точками* стратегически важные области на теле объекта, такие как суставы и конечности, которые позволяют отслеживать движения и позы объекта [1].

Ограничивающим прямоугольником будем называть область, очерчивающую границы объекта, указывающая его местоположение и размер.

Под *паттернами поведения* будем понимать характерные движения или позы, которые можно выявить на основе анализа ключевых точек и координат ограничивающего прямоугольника [1].

Говоря про *шум/шумные данные*, мы будем иметь введу данные, содержащие случайные помехи или ошибки, которые могут затруднить их точную интерпретацию или анализ [5].

В статье рассматривается проблема классификации паттернов поведения лабораторных крыс на основе анализа последовательности видеок кадров [1]. Введем понятие *walking*, так будем называть дальше класс крыса ходит/сидит. Для решения задачи классификации класса мы использовали одномерный фильтр Калмана с постоянной скоростью [6], усреднение результатов классификации [7] среди пяти кадров и медианный фильтр [8]. Классификация происходит по вектору скорости [9] и пороговому значению по длине этого вектора. Введем понятие *rearing*, так будем называть дальше класс крыса встала на задние лапы. Для решения задачи классификации этого класса мы использовали модель машинного обучения

Random Forest [10] и усреднение результатов классификации [7] среди пяти кадров.

В данной статье также рассматривается задача классификации кадров в потоковом режиме, где кадры классифицируются поочерёдно, один за другим. Решение этой задачи формализовано в алгоритме программы [11]. Вместо выполнения каждого этапа сразу на всех данных, алгоритм реализует последовательность действий для классификации каждого кадра, что позволяет получать результаты классификации в режиме реального времени, не дожидаясь завершения обработки всех данных. Задача переводится в дискретную форму [12], обработка данных происходит поэтапно, где классификация нового кадра начинается после окончания классификации предыдущего, обеспечивая потоковую классификацию.

Аналогичное решение было предложено в работе [13]. В этой работе представлена компьютерная система распознавания образов, которая упрощает задачу анализа поведения животных, сводя к минимуму необходимость вмешательства человека. Система использует две видеокамеры для съёмки поведения крыс в горизонтальной и вертикальной плоскостях в течение 15 минут. Их поведение фиксируется со скоростью один кадр в секунду. Данные с видеокамер преобразуются в формат, приемлемый для компьютеров Micro Vax I и VAX 11/750. Каждое видеоизображение преобразуется в массив размером 256 на 256, и в итоге каждый 15-минутный сеанс наблюдения генерирует 28800 блоков информации по 512 байт каждый. Используя математически полный набор моментов до четвёртого порядка и связанные с ними скалярные инварианты, компьютер запрограммирован на распознавание пяти основных положений тела крысы, включая стоячее, сидячее, на задних лапах, ходячее и лежащее. Компьютер также запрограммирован на распознавание таких действий, как уход за шерстью, поворот головы, поворот всего тела, взгляд, обоняние, обнюхивание и умывание. Эта система не только ускоряет классификацию поведения, но и

устраняет субъективность, часто критикуемую при участии наблюдателей-людей.

3 Классификация движения крысы

Для описания процесса анализа поведения крыс введем следующие обозначения и понятия на языке математики.

Ограничивающий прямоугольник очерчивает границы объекта на видеокадре [2]. Обозначим его как $bbox = (x_1, y_1, x_2, y_2)$, где (x_1, y_1) и (x_2, y_2) – координаты левого верхнего и правого нижнего углов соответственно. Изменение положения ограничивающего прямоугольника во времени, обозначается как $\vec{v} = (v_x, v_y)$ и назовем *вектором скорости*.

Рассмотрим понятие *Фильтр Калмана* [6]. Он используется для предсказания и коррекции координат и размеров ограничивающего прямоугольника на основе предыдущих измерений. Обозначим его как *kalman*. Фильтр включает в себя 6 состояний и 4 измерения. Под *состоянием системы* будем понимать состояние фильтра Калмана, который на каждом шаге описывается вектором состояния $X = [x, y, w, h, v_x, v_y]^T$, где x, y – координаты центра *ограничивающего прямоугольника*; w, h – ширина и высота; v_x, v_y – компоненты скорости. Под *измерением системы* будем понимать измерения фильтра Калмана, которые на каждом шаге описывается уравнением измерения $z_k = [x_k, y_k, w_k, h_k]^T = H_k X_k + R_k$, где z_k – вектор измерений на шаге k ; H_k – матрица измерений; $\vec{X}_k$ – вектор состояния системы на шаге k ; R_k – шум измерений, задается через ковариационную матрицу шума измерений.

Под *пороговым значением*, которое обозначим как *threshold*, будем понимать минимальное значение длины вектора скорости, при котором можно утверждать, что крыса ходит. Логическую переменную, указывающую, находится ли крыса в движении назовем *is_walking*. Одномерный массив *History*, будет содержать результаты классификации для последних 10 кадров. Он используется для усреднения и сглаживания текущих значений.

3.1 Математическая модель метода классификации движения крысы

Дано: Видеопоследовательность, состоящая из N кадров, каждый из которых содержит координаты *ограничивающего прямоугольника*, описывающие положение крысы [2]. Также задается пороговое значение *threshold* для классификации поведения крысы на основе вектора скорости. Кроме того, начальные шесть кадров используются для усреднения координат и размеров *ограничивающего прямоугольника*, что позволяет установить начальное состояние фильтра Калмана $\bar{x} = \frac{1}{6} \sum_{i=1}^6 x_i, \bar{y} = \frac{1}{6} \sum_{i=1}^6 y_i, \bar{\omega} = \frac{1}{6} \sum_{i=1}^6 \omega_i, \bar{h} = \frac{1}{6} \sum_{i=1}^6 h_i$ [6]. Сам фильтр Калмана задаётся 4 матрицами:

1. *Матрица перехода* (Transition Matrix) представляет вероятности перехода от одного шага к другому, описывая изменение состояния системы [6]. В контексте моделирования движения объекта с постоянной скоростью, мы используем матрицу перехода, чтобы показать, как координаты и размеры *ограничивающего прямоугольника* изменяются в зависимости от скорости. Единицы на диагонали матрицы указывают на то, что при нулевой скорости координаты и размеры *ограничивающего прямоугольника* остаются неизменными. Единицы в строках, соответствующих скорости (v_x и v_y), в матрице перехода указывают на то, что изменение координат объекта происходит линейно в зависимости от скорости. Это позволяет нам моделировать движение объекта таким образом, что при каждом шаге координаты x и y изменяются на величину, равную соответствующей компоненте скорости [6].

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

2. *Матрица измерений* (Measurement Matrix) связывает состояние системы с измерениями, определяя, какие компоненты состояния могут быть непосредственно измерены [6]. Единицы на диагонали указывают на

возможность измерения координат и размеров ограничивающего прямоугольника (x, y, w, h) . Компоненты скорости (v_x, v_y) не могут быть измерены напрямую, что требует использования косвенных методов для их оценки.

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

3. *Ковариационная матрица шума процесса* (Process Noise Covariance Matrix) описывает неопределенность или шум [5] в модели процесса, показывая, насколько сильно состояние системы может отклоняться от предсказанного значения из-за внутренних факторов [6]. В рассматриваемой задаче вектор состояния имеет размерность 6 и включает координаты (x, y) , размеры (w, h) и скорости (v_x, v_y) объекта. Таким образом, размерность матрицы Q равна 6×6 . Мы будем считать, что компоненты вектора состояния не коррелируют друг с другом. В этом случае можно использовать упрощённую модель. Матрица Q будет являться диагональной матрицей, где все элементы вне главной диагонали равны нулю. Диагональные элементы представляют собой дисперсии шума (σ^2) для каждого параметра состояния. Дисперсия рассчитывается как квадрат стандартного отклонения погрешности детектора, будем считать, что для всех диагональных элементов она одна. Для моделирования реалистичного сценария уровень шума оценён экспериментально: стандартное отклонение ошибки детектирования принято равным 0.13 пикселя, что соответствует дисперсии $\sigma^2 = 0.017$ для всех измеряемых параметров. Таким образом, матрица Q задаётся как $Q = 0.017 \cdot I_{6 \times 6}$, где $I_{6 \times 6}$ — единичная матрица.

$$Q = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \times 0.017$$

4. Ковариационная матрица шума измерений (Measurement Noise Covariance Matrix) [6] характеризует дисперсию погрешностей данных, получаемых от системы детектирования [5]. В рассматриваемой задаче вектор измерений имеет размерность 4 и включает координаты и размеры объекта (x, y, w, h). Таким образом, размерность матрицы R равна 4×4 . Мы будем считать, что компоненты вектора измерений не коррелируют друг с другом. В этом случае можно использовать упрощённую модель. Матрица R будет являться диагональной матрицей, где все элементы вне главной диагонали равны нулю. Диагональные элементы представляют собой дисперсии шума (σ^2) для каждого компонента измерения. Дисперсия рассчитывается как квадрат стандартного отклонения погрешности детектора, будем считать, что для всех диагональных элементов она одна. Для моделирования реалистичного сценария уровень шума оценён экспериментально: стандартное отклонение ошибки детектирования принято равным 3.16 пикселя, что соответствует дисперсии $\sigma^2 = 10$ для всех измеряемых параметров. Таким образом, матрица R задаётся как $R = 10 \cdot I_{4 \times 4}$, где $I_{4 \times 4}$ – единичная матрица

$$R = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times 10$$

Тогда необходимо найти:

1. Предсказание, коррекцию координат и размеров ограничивающего прямоугольника на каждом кадре с использованием фильтра Калмана [6]. Это включает обновление состояния системы на основе текущих измерений $x_k = x_{k-1} + K_k(z_k - H x_{k-1})$, где x_k – обновленное состояние, K_k – коэффициент усиления Калмана, z_k – измерения на шаге k .

2. Классификацию поведения крысы на основе вектора скорости $\vec{v}$ [9]:

$$is_{walking} = \begin{cases} 1, & \text{если } \|\vec{v}\| > threshold \\ 0, & \text{иначе} \end{cases}$$

3. Усреднение 5 последних значений для сглаживания результатов классификации [7]:

$$history_{k-2} = \frac{history_k + history_{k-1} + history_{k-2} + history_{k-3} + history_{k-4}}{5}$$

4. Результат применения медианного фильтра [8] к истории из 10 последних результатов классификации $history = median(history)$.

3.2 Алгоритм метода классификации движения крысы

Для решения задачи классификации был разработан алгоритм, представляющий собой класс, включающий три метода [9, 11]. Метод `filter_step()` выполняет шаг фильтрации с коррекцией на основе полученных измерений [6]. Метод `clsf_step()` осуществляет классификацию по вектору скорости [9]. Метод `step()` вызывается извне и внутри себя последовательно вызывает два предыдущих метода, возвращая отфильтрованные координаты ограничивающего прямоугольника на последнем кадре, результат классификации и вектор скорости [11]. Таким образом, задача переводится в дискретную форму [12], позволяя получать результаты классификации в потоковом режиме [11]. Дальнейшее описание алгоритма будет посвящено детальному рассмотрению каждого из этих методов и их взаимодействию в процессе обработки данных.

1 Инициализация

1.1 Входные данные: Передаются первые 6 кадров с координатами ограничивающего прямоугольника и размеры кадра (ширина и высота) [6].

1.2 Усреднение начальных данных: Рассчитываются усредненные координаты и размеры ограничивающего прямоугольника для установки начального состояния фильтра Калмана [6].

1.3 Инициализация фильтра Калмана:

- Задаются матрицы перехода, измерений, шума процесса и шума измерений [6].
- Устанавливается начальное состояние фильтра Калмана с использованием усредненных координат ограничивающего прямоугольника [6].

2 Шаг фильтрации (`filter_step`)

2.1 Входные данные: Текущий ограничивающий прямоугольник из видеокадра [2].

2.2 Подготовка измерений: Преобразование координат ограничивающего прямоугольника в формат, совместимый с фильтром Калмана [6].

2.3 Коррекция состояния: Использование текущих измерений для корректировки состояния фильтра Калмана [6].

2.4 Предсказание состояния: Вычисление предсказанных координат ограничивающего прямоугольника и вектора скорости (V_x , V_y) на основе текущего состояния фильтра Калмана [6].

3 Классификация поведения (clsf_step)

3.1 Входные данные: Вектор скорости (V_x , V_y) [9].

3.2 Вычисление скорости: Рассчитывается величина вектора скорости.

3.3 Сравнение с порогом:

- Если скорость больше порогового значения (threshold), крыса движется (is_walking = 1).
- Иначе крыса стоит на месте (is_walking = 0).

3.4 Сглаживание классификации:

- Сохраняются результаты классификации в историю (classification_history).
- Применяется усреднение среди 5 соседних значений и медианный фильтр для устранения шумов [7].

4 Основной цикл обработки кадров (step)

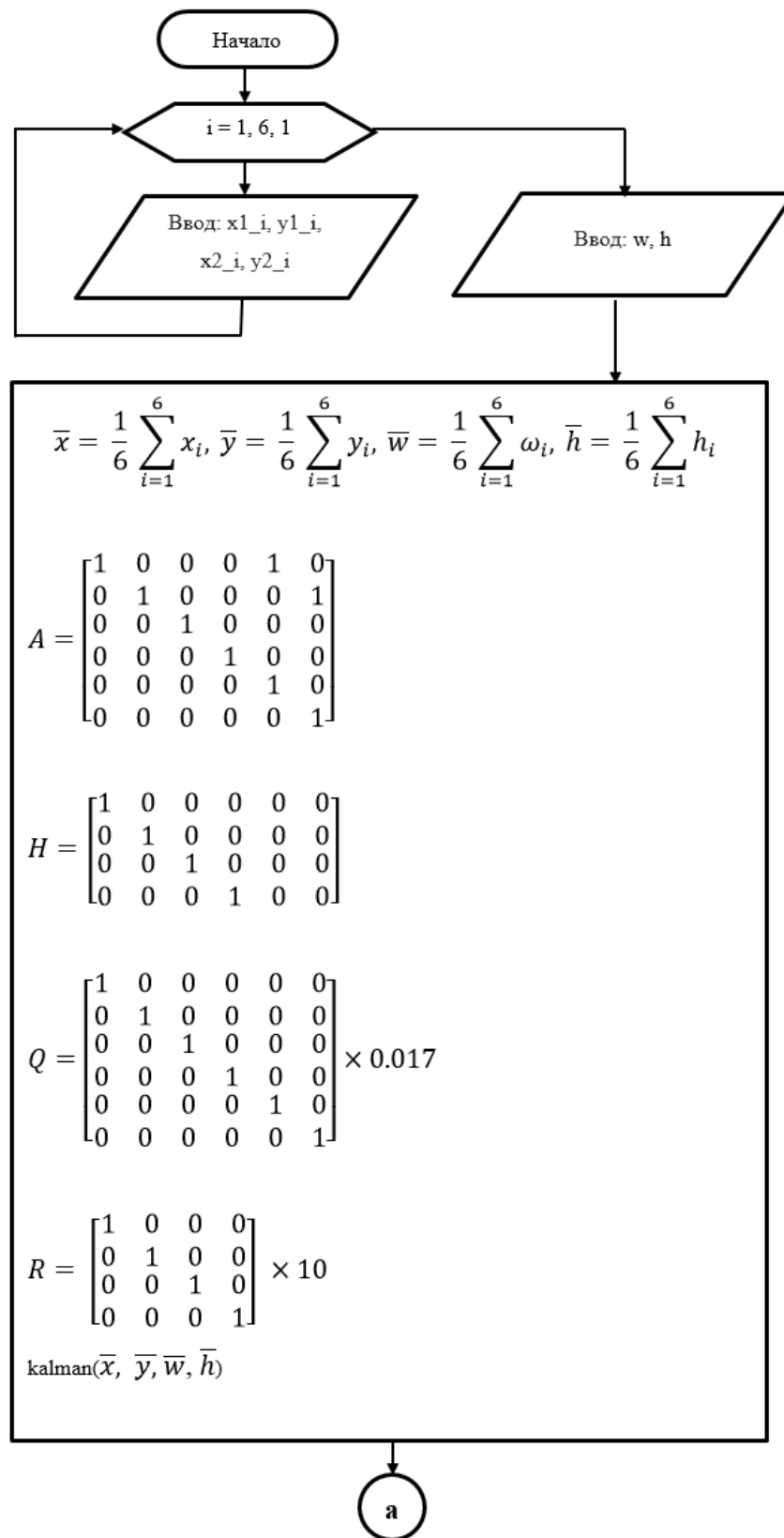
4.1 Входные данные: ограничивающий прямоугольник текущего кадра.

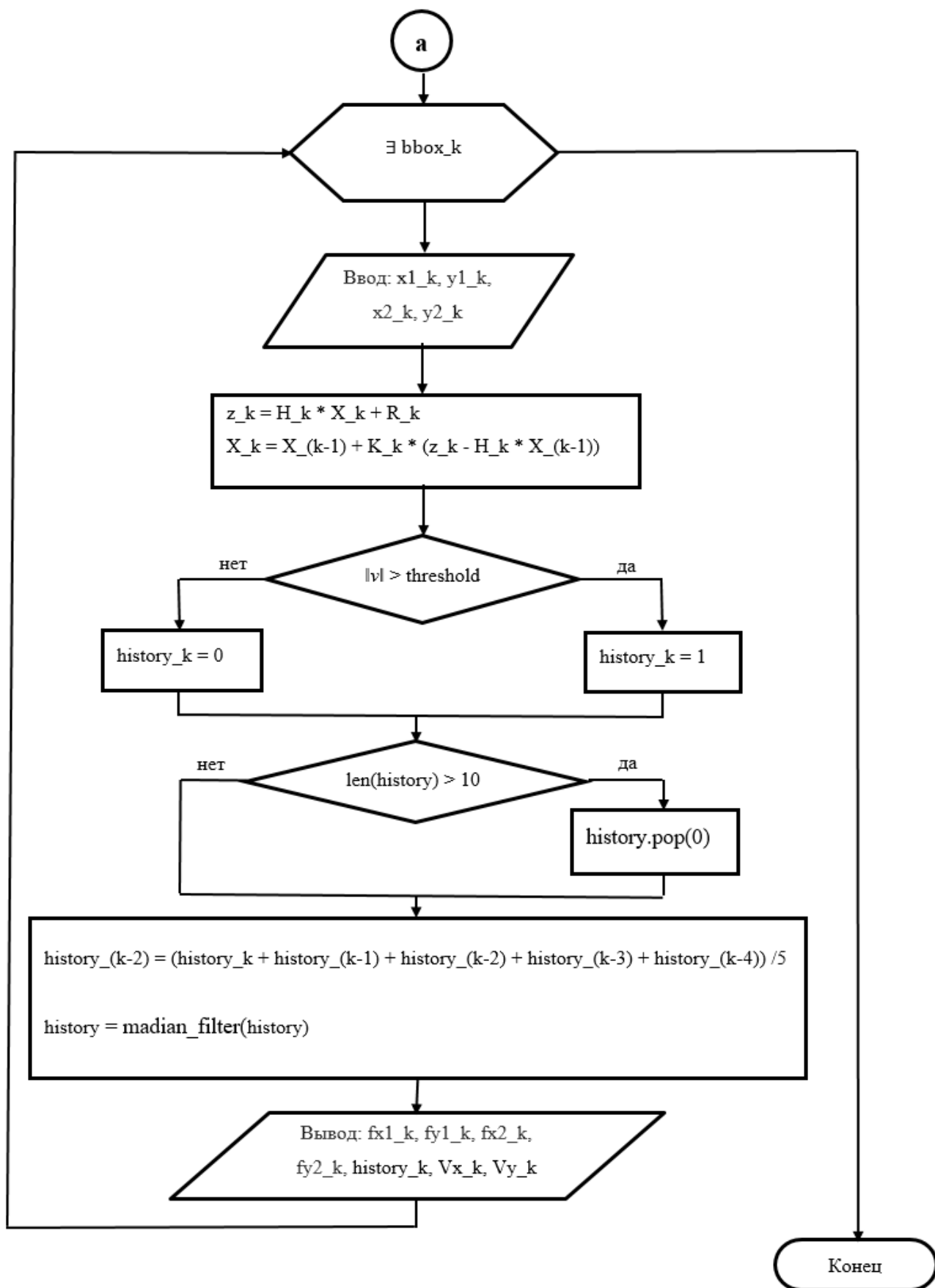
4.2 Для каждого кадра [11, 12]:

- Выполняется шаг фильтрации (filter_step), возвращающий отфильтрованный ограничивающий прямоугольник и вектор скорости.
- Выполняется шаг классификации (clsf_step), возвращающий результат классификации.

4.3 Возвращаются отфильтрованный ограничивающий прямоугольник, результат классификации и вектор скорости для текущего кадра.

Составим схему работы данного алгоритма (рис. 1 а, б)





б)

Рис. 1. а, б - Схема работы алгоритма

3.4 Эксперимент по классификации движения крысы

Для реализации класса будем использовать язык программирования python [14] и редактор кода Visual Studio Code [15] [Приложение]. Для экспериментов по подбору лучших методов определения класса и параметров для них будет использоваться Jupyter Notebook [16].

Для работы с классификатором, его тестирования и доработки, был создан новый класс [Приложение]. Данный класс предназначен для считывания из директории видеофайлов в формате .mp4 и файлов разметки в формате .txt [2]. В первой строке текстового файла содержатся данные о размере кадра, в последующих строках содержится информация о разметке кадров. Каждый кадр описывается в отдельной строке, включая номер кадра, координаты ограничивающего прямоугольника, координаты контрольных точек и их score [17]. Score представляет собой числовое значение, отражающее уверенность в определении контрольной точки, он варьируется от 0 до 1. Программа инициализирует фильтр Калмана [6], покадрово подает координаты ограничивающего прямоугольника в классификатор [2, 6, 7, 8, 9, 12] и визуализирует на видео результат классификации, контрольные точки, ограничивающий прямоугольник и вектор скорости для каждого кадра [11], а затем сохраняет результат в указанной директории (рис. 2).

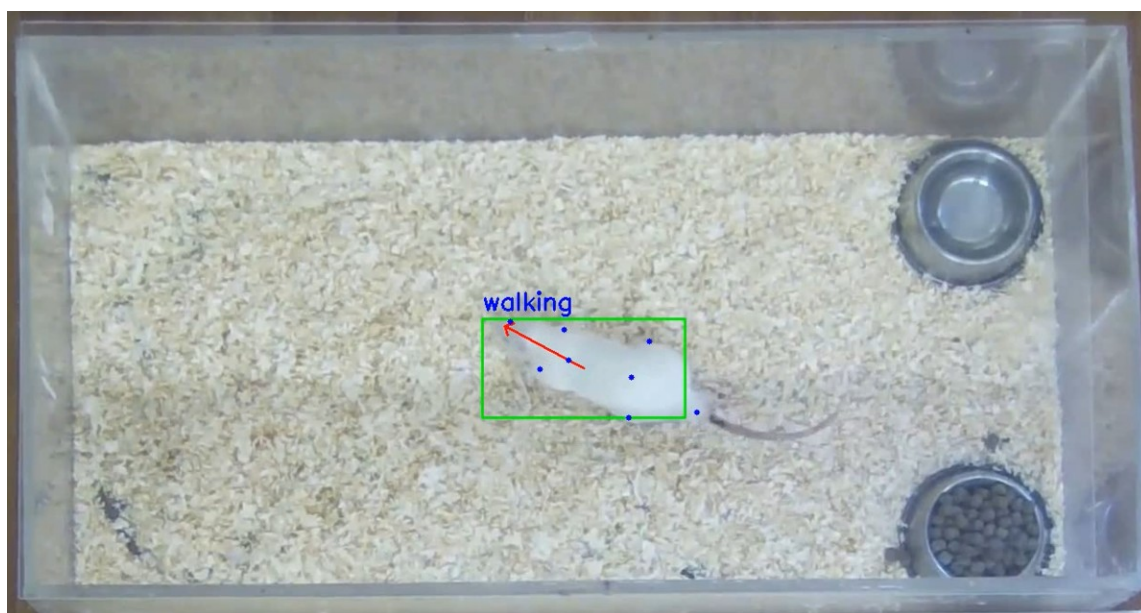


Рис. 2. Визуализация работы класса

В ходе экспериментов мы попробовали определять движение по одновременному смещению двух противоположных сторон ограничивающего прямоугольника [18]. Также попробовали использовать метрику *IoU* (Intersection over Union) [19]. Лучший результат для *IoU* был достигнут при значении 0.8, однако он оказался хуже, чем при использовании смещения ограничивающего прямоугольника. В ходе дальнейших экспериментов было установлено, что фильтр Калмана хорошо оценивает вектор скорости. Было принято решение задать пороговое значение и классифицировать движение по длине вектора скорости [9]. Это оказалось лучшим решением.

Далее стояла задача подбора параметров для алгоритма [6, 9]. В ходе эксперимента мы подобрали порог для величины вектора скорости, максимизировали коэффициент ковариационной матрицы шума процесса и минимизировали коэффициент ковариационной матрицы шума измерений [6, 9]. Комбинации параметров тестировались на различных данных с помощью написанной программы, которая автоматизировала процесс тестирования большого числа комбинаций параметров. Программа визуализировала результат классификации для каждой комбинации на одном графике, и при просмотре видео покадрово можно было исключить комбинации, которые плохо справлялись с задачей. В результате анализа удалось подобрать лучшие параметры: порог для величины вектора скорости равен 2 [9], коэффициент ковариационной матрицы шума процесса равен 0.017, коэффициент ковариационной матрицы шума измерений равен 10 [6].

В результате проделанной работы удалось добиться хорошего результата: алгоритм может быстро реагировать на изменения в поведении крысы. Он способен определять длинные и короткие промежутки активности, а также быстрые и медленные движения. Алгоритм не реагирует на повороты головы крысы и может определить движение по одному шагу. Как и любой алгоритм, он не лишён недостатков и имеет ложные срабатывания, но их количество удалось свести к минимуму.

4 Классификация паттерна: крыса встала на задние лапы

Попробуем определить класс *rearing*, когда крыса встаёт на задние лапы. Нам нужно найти признаки, по которым можно определить данный паттерн поведения [1]. Для этого напомним класс, который будет принимать координаты контрольных точек, результат классификации класса *walking* и вектор скорости для каждого видео.

Контрольные точки [2] на теле крысы расставлены следующим образом: (0) – нос, (1, 2) – уши, (3) – точка между лопатками, (4) – середина позвоночника, (5, 6) – задние конечности, (7) – начало хвоста. Визуализируем расстановку точек (рис. 3).

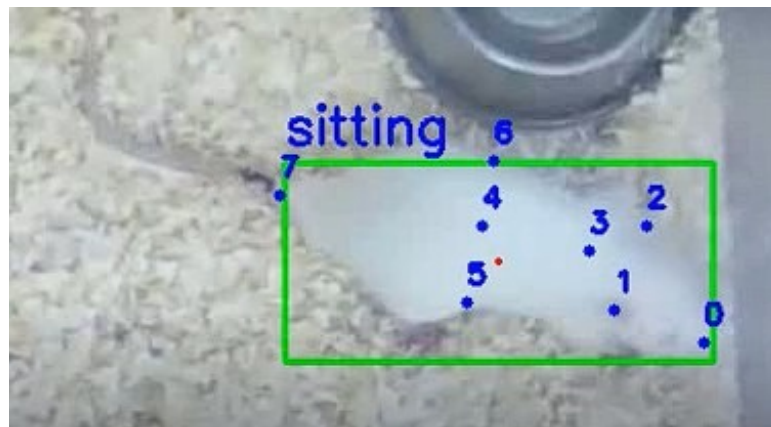


Рис. 3. Расстановка контрольных точек

Введем понятие *точка a*, *точка b*, когда будем говорить о комбинации из двух точек [2] для расчёта некоторых признаков.

4.1 Математическая модель метода классификация паттерна: крыса встала на задние лапы

Дано: набор контрольных точек [2] и координаты вектора скорости для двух последних кадров. Кроме того, имеется частота кадров в видео и результат классификации класса "walking".

Тогда нам необходимо найти признаки [1], по которым мы могли бы определить класс:

1. Углы между прямыми [1], образованными двумя точками (0 – 3, 3 – 4, 4 – 7, 0 – 7), на разных кадрах. Для вычисления мы используем функцию

$\arctan2$ [20]. Эта функция позволяет вычислить угол между двумя векторами. Arctan2 , в отличие от обычного $\arctan$, учитывает знак обоих аргументов, чтобы определить правильный квадрант для угла. Это позволяет корректно определять угол в полном диапазоне от $-\pi$ до π (от -180° до 180°), в то время как $\arctan$ возвращает значения только в диапазоне от $-\pi/2$ до $\pi/2$. Для описания направления и величины изменения расстояния между двумя точками [21] в предыдущем кадре будем использовать $v_prev_k = (x_{a,(k-1)} - x_{b,(k-1)}, y_{a,(k-1)} - y_{b,(k-1)})$, а для описания направления и величины изменения расстояния между теми же двумя точками [21], но уже в текущем кадре, будем использовать $v_curr_k = (x_{a,k} - x_{b,k}, y_{a,k} - y_{b,k})$. В результате мы сможем посчитать угол между прямыми $\text{Angle}_{a-b,k} = \arctan2(\det[v_prev_k, v_curr_k], v_prev_k \cdot v_curr_k)$ [22].

2. *Смещения положения точек (0, 3, 4, 7) между кадрами.* Для расчёта будем использовать евклидово расстояние [23] между точкой на текущем и предыдущем кадре $\text{Distance}_a = \sqrt{(x_k - x_{k-1})^2 + (y_k - y_{k-1})^2}$.

3. *Расстояния между 2 ключевыми точками (0 – 3, 3 – 4, 4 – 7, 0 – 7) на одном кадре.* Для расчетов будем использовать формулу евклидова расстояния [23] между двумя точками $\text{distances_dict}_{a-b} = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}$.

4. *Скорость перемещения точек (0, 3, 4, 7) на кадре.* Для расчетов будем использовать формулу скорости $\text{speed}_a = \frac{\text{Distance}_a}{\frac{1}{\text{fps}}}$.

5. *Изменение в направлении вектора скорости.* Для расчётов будем использовать формулу косинусную меру угла между векторами [24] $\text{velocity_change} = \arccos\left(\frac{\vec{v}_{k-1} \cdot \vec{v}_k}{\|\vec{v}_{k-1}\| \|\vec{v}_k\|}\right)$, формула вычисляет угол между двумя векторами с использованием скалярного произведения и нормы векторов. Для аргумента функции $\arccos$ зададим ограничения в пределах от -1.0 до 1.0 , чтобы избежать численных ошибок, связанных с выходом за пределы

допустимого диапазона. Формула вычисляет угол между двумя векторами в диапазоне от 0 до π радиан (или от 0° до 180°).

Сформируем набор данных для предсказания класса. Все вычисленные значения (углы, расстояния, скорости, изменения вектора скорости), а также результат классификации класса walking соберём в DataFrame. Пустые значения заполним нулями.

4.2 Алгоритм метода классификация паттерна: крыса встала на задние лапы

Для решения задачи классификации был разработан алгоритм, представляющий собой класс [11], включающий три метода. Метод `data_step()` извлекает признаки из координат ключевых точек [1, 2, 20, 21, 22, 23, 24], вектора скорости и результатов классификации класса walking. Метод `clsf_step()` классифицирует поведение крысы с использованием модели Random Forest [10] и усредняет [7] текущее значение среди двух предыдущих и двух последующих кадров. Метод `step()` вызывается извне и внутри себя последовательно вызывает два предыдущих метода [11], возвращая результат классификации класса rearing. Таким образом, задача переводится в дискретную форму [12], позволяя получать результаты классификации в потоковом режиме. Дальнейшее описание алгоритма будет посвящено детальному рассмотрению каждого из этих методов и их взаимодействию в процессе обработки данных.

1. Инициализация

1.1 Входные данные:

- Загружается модель машинного обучения (`random_forest_model.joblib`) [10].
- Инициализируется нормализатор (`StandardScaler`) [25].

1.2 Усреднение начальных данных:

- Создается пустой список `classification_history` для хранения истории классификаций.

2. Шаг подготовки данных (data_step)

2.1 Входные данные:

- Словарь ключевых точек (keypoints_dict) [2], частота кадров (fps), векторы скорости (velocity_vectors), результаты классификации (classification_results).

2.2 Подготовка данных:

- Вычисляются углы между прямыми, образованными ключевыми точками [1, 20, 21, 22].
- Смещение положения точек [1, 23].
- Вычисляются расстояния между парами ключевых точек [1, 23].
- Вычисляются скорости перемещения точек между кадрами.
- Вычисляются изменения угла вектора скорости [24].
- Формируется DataFrame с подготовленными данными для предсказания модели.

3. Шаг классификации поведения (clsf_step)

3.1 Входные данные:

- Подготовленные данные (data).

3.2 Классификация поведения:

- Модель Random Forest [10] предсказывает результат классификации.
- Результат сохраняется в classification_history.
- Возвращается усредненное значение последних 5 результатов классификации.

4. Основной цикл обработки кадров (step)

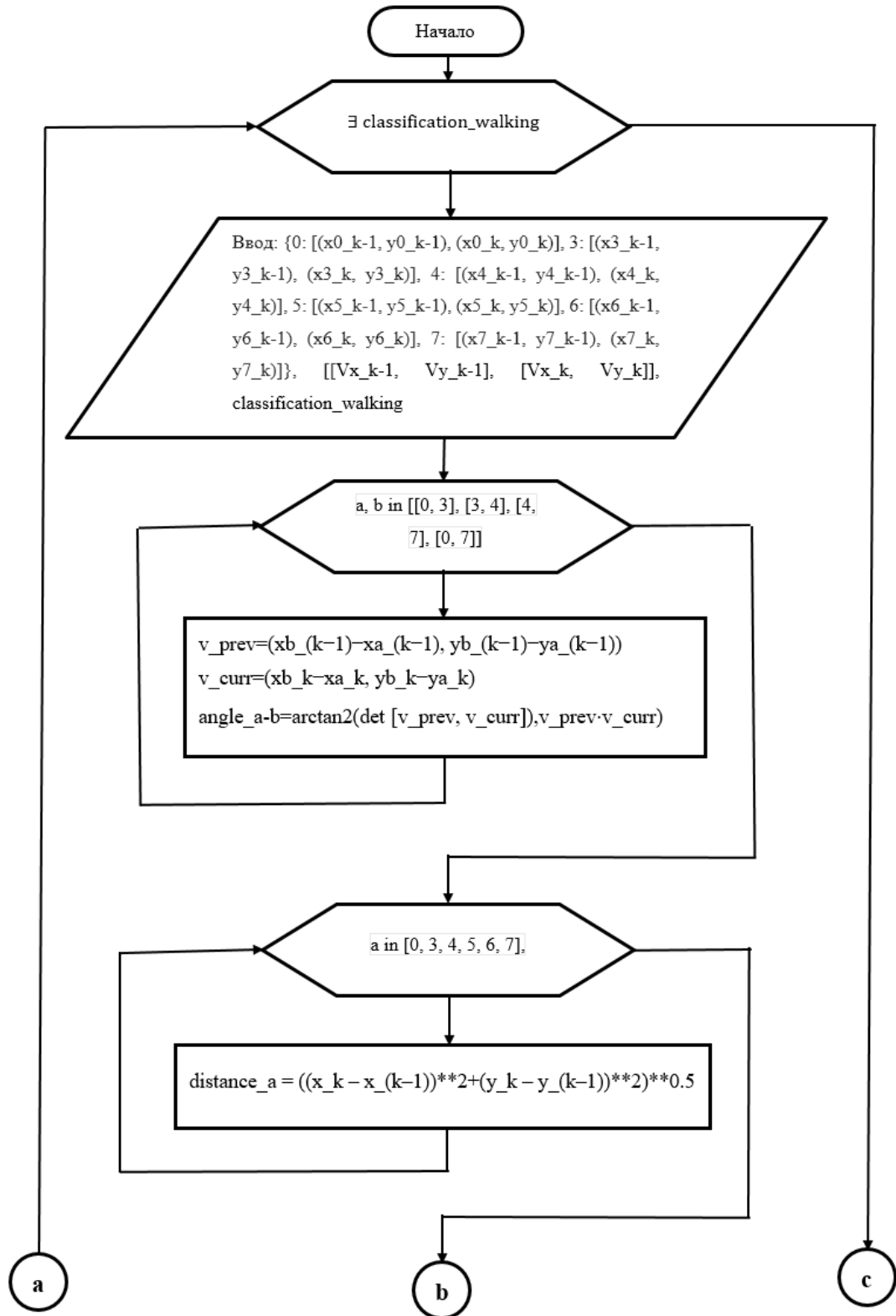
4.1 Входные данные:

- словарь ключевых точек (keypoints_dict)
- частота кадров (fps)
- вектор скорости (velocity_vector)
- результаты классификации (classification_results)

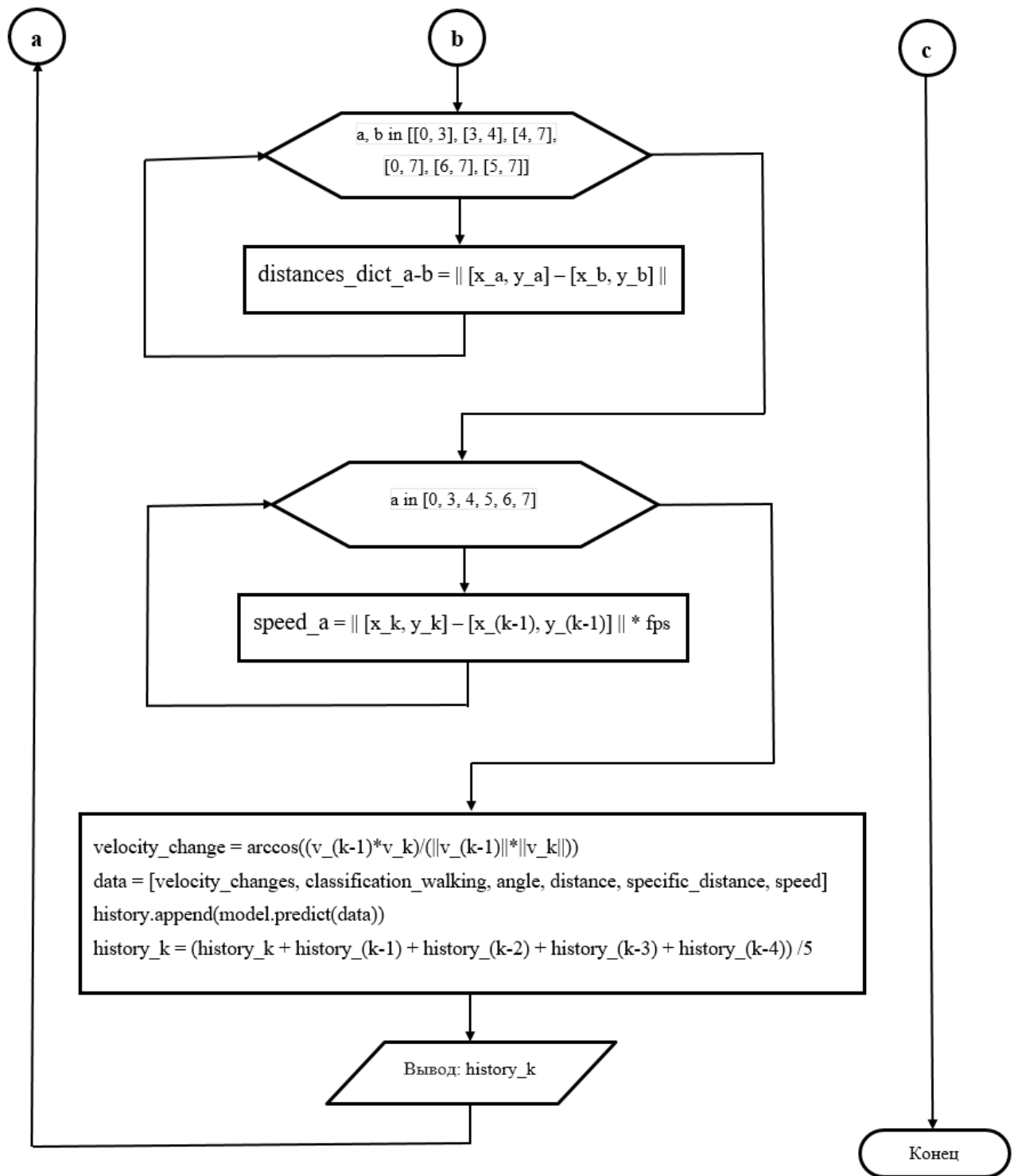
4.2 Обработка кадров:

- Для каждого кадра выполняется подготовка данных (data_step) [1]
- предсказание (clsf_step) [10].
- Возвращается результат классификации для текущего кадра.

Составим схему работы данного алгоритма (рис. 4)



a)

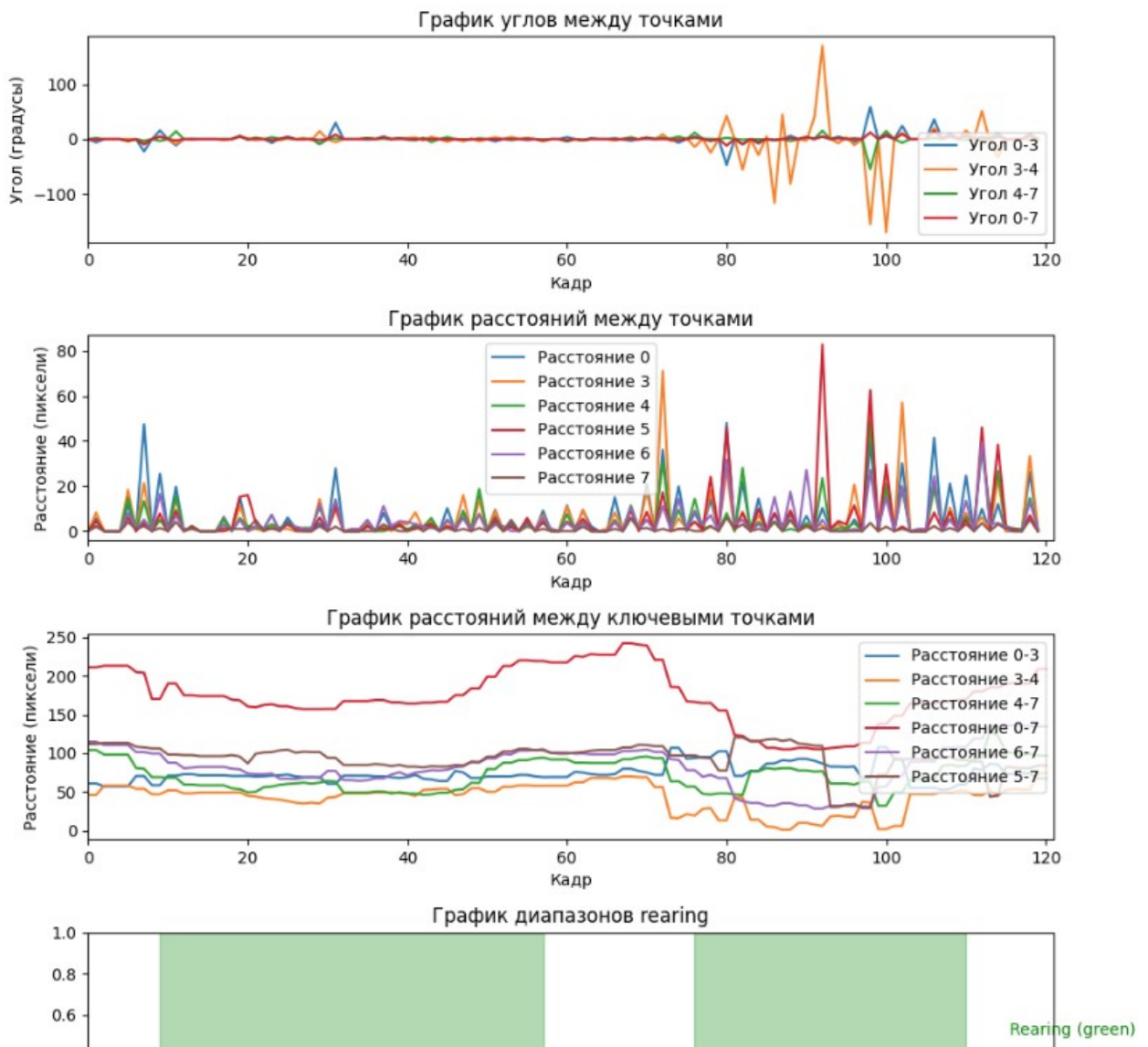


б)

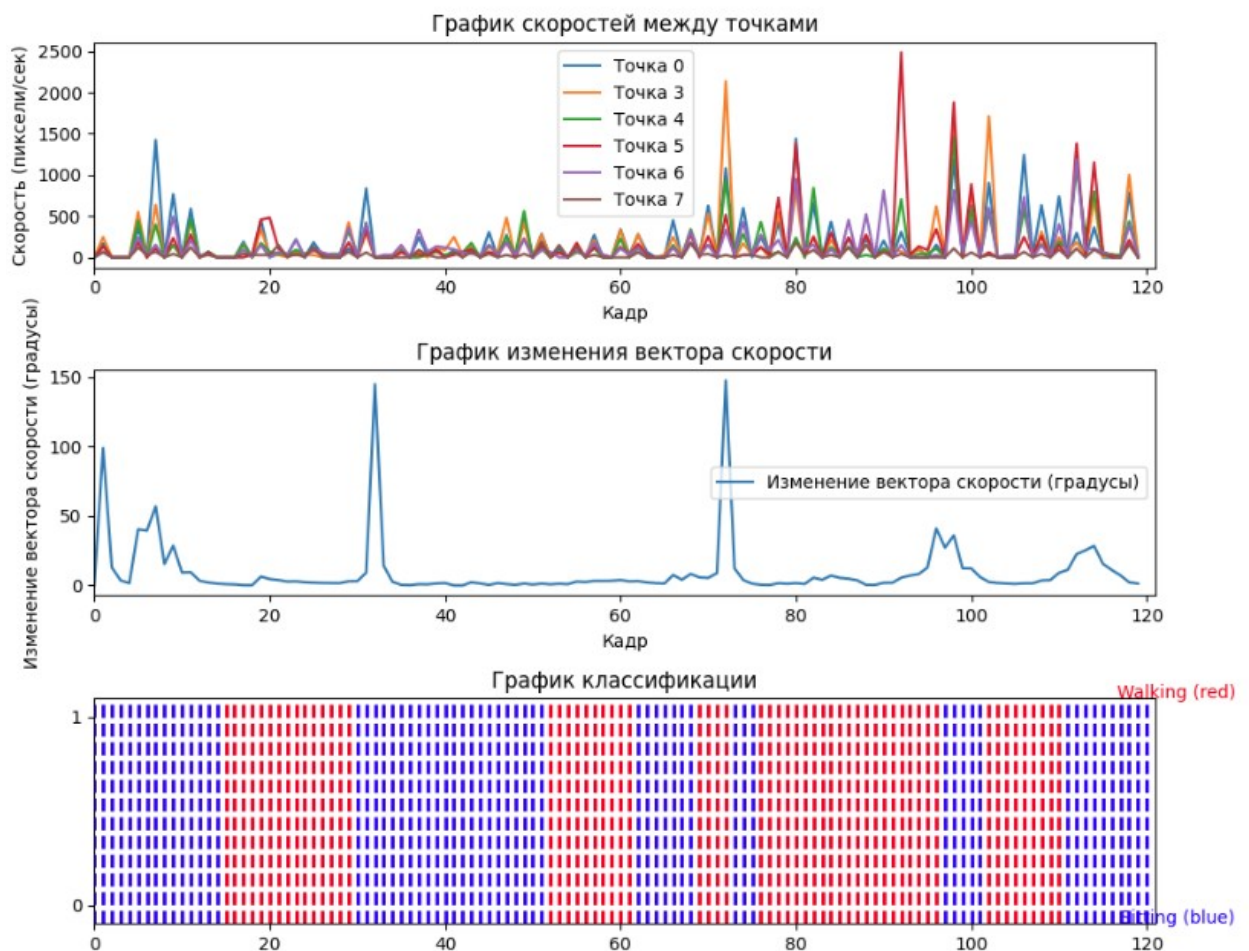
Рис. 4. а, б - Схема работы алгоритма

4.3 Эксперимент по классификации паттерна: крыса встала на задние лапы

Извлечём признаки из контрольных точек [2]: угол между прямыми [1, 20, 21, 22], образованными ключевыми точками 0 – 3, 3 – 4, 4 – 7, 0 – 7; смещение точек 0, 3, 4, 5, 6, 7 [1, 23]; расстояние между точками 0 – 3, 3 – 4, 4 – 7, 0 – 7, 6 – 7, 5 – 7 [1, 23]; скорость точек 0, 3, 4, 5, 6, 7; изменение угла вектора скорости. Объединим новые признаки с результатом классификации класса walking. Разметим каждый кадр набора данных gearing, отметив кадры, на которых крыса стоит на задних лапах, и визуализируем данные на графиках для каждого видео (рис. 5).



а)



б)

Рис. 5. а, б - Пример визуализация признаков на одном видео

Попробуем найти закономерности и выдвинем гипотезы как можно определить класс *rearing* [1]:

- Когда падает расстояние между крайними точками (0 – 7) на теле крысы. Но также это может удовлетворять классу крыса сворачивается в клубочек.
- Когда сильно меняется направление вектора скорости, а смещение точки (7) хвост меняется слабо.
- Когда меняется резко угол направления, куда смотрит мордочка (0-3), а смещение точки хвост (7) остается практически без изменений. Так как крыса делает резкое движение чтобы подняться на задние лапы.

- Предположим, что, когда крыса стоит на 4 лапах, тогда расстояние между ключевыми точками в убывающей последовательности выглядит так: от середины спины до хвоста (4 – 7), мордочка (0 – 3), от мордочки до середины спины (3 – 4). А на 2 лапах так: мордочка (0 – 3), от середины спины до хвоста (4 – 7), от мордочки до середины спины (3 – 4). То есть зона мордочки становится больше, а зона конца туловища меньше.

Подготовим набор данных из признаков по которым строились графики. Чтобы проверить гипотезы напишем программы [14, 15, 16], которые будут выводить номера кадров на которых соблюдаются условия из гипотезы. По результатам выводится большое количество кадров и плохо отделяется наш класс $rearing = 1$ от $rearing = 0$. *Recall* (полнота) – это доля истинно положительных результатов среди всех фактически положительных случаев, показывает, насколько хорошо алгоритм находит все положительные примеры. *Precision* (точность) – это доля истинно положительных результатов среди всех положительных прогнозов, показывает, насколько точны положительные прогнозы. *Recall* получился хорошим, а *precision* маленьким [26] из-за того, что в положительный класс попало много значений, как истинно положительных, так и ложноположительных. Это говорит, что алгоритм плохо описывает $rearing = 1$.

Попробуем обучить на всех признаки модель *RandomForest* [10], а потом посмотрим метрику *Feature Importance* [28]. *Feature Importance* (важность признаков) – это мера, показывающая, насколько каждый признак влияет на предсказание модели. Будем использовать комбинации признаков. Где на каждом признаке обучим свое дерево решений [10]. А результат будет определиться путем взвешенного голосования [27], по результатам метрики *Feature Importance* [28] (рис. 6).

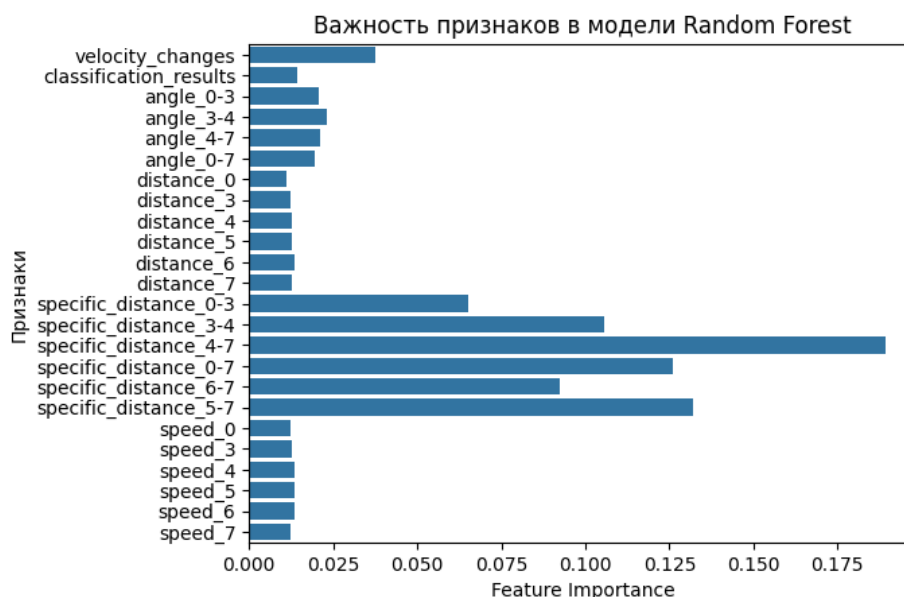


Рис. 6. Важность признаков

Лучшей комбинацией признаков оказалось использование расстояния между точками: 0 – 3, 3 – 4, 4 – 7, 0 – 7, 6 – 7, 5 – 7 и значение изменение угла вектора скорости. Что подтверждает полученный график важности признаков в модели Random Forest (рис. 6) [28], который говорит о том, что модель в основном опирается на расстояние между точками при классификации. Но даже при лучшей комбинации модель выдает плохой результат. Метрики с результатами работы модели сведены в Таблицу 1 [26]. Таблица состоит из нескольких колонок. Колонка *precision* (точность) показывает долю истинно положительных результатов среди всех положительных прогнозов, что отражает точность модели в предсказании положительного класса. Колонка *recall* (полнота) отображает долю истинно положительных результатов среди всех фактически положительных случаев, демонстрируя способность модели находить все положительные примеры. *f1-score* представляет собой среднее гармоническое между *precision* и *recall*, предоставляя баланс между этими двумя метриками и показывая общую эффективность модели. Колонка *support* указывает количество истинных примеров в каждом классе, что важно для понимания распределения данных. Модель пытается различить класс *rearing* = 0, когда крыса стоит на четырёх лапах, и класс *rearing* = 1, когда крыса стоит на

двух задних лапах. *Accuracy* (точность) показывает долю правильно предсказанных примеров среди всех примеров, давая общее представление о точности модели. *Macro avg* и *weighted avg* предоставляют средние значения метрик, где *macro avg* рассчитывается как простое среднее, не учитывающее размер классов, а *weighted avg* учитывает размер каждого класса, что важно при наличии дисбаланса классов.

Таблица 1 – Метрики модели Random Forest, обученной на значимых признаках

	precision	recall	f1-score	support
rearing = 0	0.47	0.51	0.49	2148
rearing = 1	0.27	0.39	0.32	1253
accuracy			0.40	3401
macro avg	0.37	0.45	0.40	3401
weighted avg	0.40	0.47	0.43	3401

Модель показывает низкие значения *precision* и *recall* для класса *rearing = 1* [26], что указывает на плохую способность модели различать этот класс. Низкий *f1-score* для *rearing = 1* подтверждает, что модель плохо справляется с классификацией этого класса. Общая точность модели составляет 40%, что является низким показателем.

Сделаем предположение о том, что класс *rearing* легче всего определить по началу класса, когда крыса делает резкое движение чтобы подняться на задние лапы [1]. Метрики с результатами работы модели сведем в Таблицу 2.

Таблица 2 – Метрики модели Random Forest, обученной определять начало класса

	precision	recall	f1-score	support
rearing = 0	1.00	1.00	1.00	3384
rearing = 1	0.20	0.06	0.09	17
accuracy			0.99	3401
macro avg	0.60	0.53	0.54	3401
weighted avg	0.99	0.99	0.99	3401

Проведем анализ результатов. Высокие значения precision и recall для rearing = 0 обусловлены дисбалансом классов [26]. Нам не удалось хорошо обучить модель, чтобы она научилась определять класс rearing = 1. Низкий f1-score для rearing = 1 подтверждает, что модель плохо справляется с классификацией этого класса. Высокая общая точность (99%) также обусловлена дисбалансом классов.

Мне не удалось найти конкретные значения признаков, по которым можно определить класс rearing = 1 [1]. Далее попробуем использовать готовую модель Random Forest [10], которую будем обучать сразу на всех признаках набора данных rearing, чтобы в дальнейшем интегрировать модель в мой классификатор. Метрики с результатами работы модели сведем в Таблицу 3.

Таблица 3 – Метрики модели Random Forest, обученной на наборе данных rearing

	precision	recall	f1-score	support
<i>rearing</i> = 0	0.57	0.83	0.67	976
<i>rearing</i> = 1	0.86	0.62	0.72	1640
accuracy			0.70	2616
macro avg	0.71	0.73	0.70	2616
weighted avg	0.75	0.70	0.70	2616

Проанализируем результаты таблицы. Улучшение метрик для обоих классов, особенно для rearing = 1 [26], указывает на лучшее качество классификации. Улучшение f1-score для обоих классов подтверждает, что модель стала работать лучше. Общая точность составляет 70%, что является хорошим результатом.

Можно заметить, что precision для класса rearing = 0 меньше чем для класса rearing = 1 [26]. Это позволяет сделать вывод, что модели недостаточно данных в наборе rearing для обучения классификации класса rearing = 0.

Попробуем расширить набор данных за счёт добавление в него данных из набора *walking*, в разметке укажем что нет ни одного кадра, где крыса встала на задние лапы. Метрики с результатами работы модели сведем в Таблицу 4 [26].

Таблица 4 – Метрики модели Random Forest, обученной на наборах данных *rearing* и *walking*

	precision	recall	f1-score	support
<i>rearing</i> = 0	0.79	0.95	0.86	2148
<i>rearing</i> = 1	0.87	0.57	0.69	1253
accuracy			0.81	3401
macro avg	0.83	0.76	0.78	3401
weighted avg	0.82	0.81	0.80	3401

В результате значительно улучшились метрики для обоих классов. Высокий f1-score также подтверждает, что модель стала работать лучше. Общая точность составляет 81%, что является лучшим результатом.

Попробуем использовать модель GradientBoosting [29]. Метрики с результатами работы модели сведем в Таблицу 5 [26].

Таблица 5 – метрики модели GradientBoosting, обученной на наборах данных *rearing* и *walking*

	precision	recall	f1-score	support
<i>rearing</i> = 0	0.76	0.94	0.84	2148
<i>rearing</i> = 1	0.83	0.49	0.62	1253
accuracy			0.78	3401
macro avg	0.79	0.72	0.73	3401
weighted avg	0.79	0.78	0.76	3401

Из таблицы можно заметить улучшение метрик для класса *rearing* = 0, но ухудшение для *rearing* = 1 [26]. Низкий f1-score для *rearing* = 1 подтверждает, что модель плохо справляется с классификацией этого класса. Общая точность составляет 78%, что ниже, чем у модели Random Forest [10].

Таким образом, проведенный эксперимент показал, что Random Forest [10], обученной на наборах данных *rearing* и *walking*, демонстрирует хорошую эффективность в определении класса *rearing*. Добавим к нему усреднение на 5 кадрах, чтобы сгладить результаты классификации модели. Оформим итоговое решение в класс, используя язык программирования python [14] и редактор кода Visual Studio Code [15] [Приложение]. Классификатор хорошо определяет класс вставание на задние лапы, давая минимальные ошибки на тестовых видео.

Заключение

В ходе исследования были разработаны и протестированы методы идентификации паттернов поведения лабораторных крыс на основе анализа видеок кадров. Для классификации класса "ходит/сидит" использовался фильтр Калмана в сочетании с медианным фильтром и усреднением результата среди 5 кадров, что позволило эффективно определять активность крыс, включая длинные и короткие промежутки движения, а также быстрые и медленные перемещения. Этот метод показал высокую точность и устойчивость к шуму с минимальным количеством ложных срабатываний.

Для классификации класса вставание на задние лапы была применена модель Random Forest с усреднением результата среди 5 кадров и обученная на двух наборах данных *rearing* и *walking*. Подход продемонстрировал лучший результат. Таким образом, предложенные методы являются эффективными инструментами для автоматизации мониторинга поведения крыс. Результаты исследования в дальнейшем помогут в выявлении паттернов поведения пациентов в больнице, для создания системы мониторинга в реальном времени за пациентами без участия врачей.

Однако, моё решение не лишено недостатков и для получения более точного результата, требуется проведение дальнейших исследований и экспериментов с другими методами, которые позволят более точно определять паттерны поведения крыс.

Список литературы

1. **Bishop C. M.**, "Pattern Recognition and Machine Learning", Springer, 2006. P. 15-30.
2. **Szeliski R.**, "Computer Vision: Algorithms and Applications", Springer, 2010. P. 45-60.
3. **Papandreou G.**, et al., "PersonLab: Person Pose Estimation and Instance Segmentation with a Body-Driven Model", CVPR, 2018. P. 120-130.
4. CVAT Documentation. [Электронный ресурс]. URL: <https://github.com/openvinotoolkit/cvat> (дата обращения: 16.02.2025).
5. **Russakovsky O.**, et al., "ImageNet Large Scale Visual Recognition Challenge", IJCV, 2015. P. 78-90.
6. **Гребенщиков, Д. С.**, "Фильтр Калмана и его применение", Физматлит, 2009. С. 56-78.
7. **Brown R. G., Hwang P. Y. C.**, "Introduction to Random Signals and Applied Kalman Filtering", Wiley, 1997. P. 89-102.
8. **Hastie, T., Tibshirani, R., Friedman, J.**, "The Elements of Statistical Learning", Springer, 2009. P. 112-134.
9. **Goodfellow I., Bengio Y., Courville A.**, "Deep Learning", MIT Press, 2016. P. 223-245.
10. **Breiman L.**, "Random Forests", Machine Learning, 2001. [Электронный ресурс]. URL: https://www.stat.berkeley.edu/~breiman/randomforest/cc_home.htm (дата обращения: 05.03.2025).
11. **Murphy K. P.**, "Machine Learning: A Probabilistic Perspective", MIT Press, 2012. P. 156-178.
12. **Hastie T., Tibshirani R., Friedman J.**, "The Elements of Statistical Learning", Springer, 2009. P. 112-134.
13. **Kernan Jr, W. J., Mullenix, P. J., & Hopper, D. L.**, (1987). Pattern recognition of rat behavior. Physiology & Behavior. [https://doi.org/10.1016/0091-3057\(87\)90367-4](https://doi.org/10.1016/0091-3057(87)90367-4) (PMID: 3659080) (дата обращения: 26.02.2025).
14. Python Software Foundation. [Электронный ресурс]. URL: <https://www.python.org> (дата обращения: 21.02.2025).
15. Visual Studio Code Documentation. [Электронный ресурс]. URL: <https://code.visualstudio.com/> (дата обращения: 21.02.2025).
16. Jupyter Notebook Documentation. [Электронный ресурс]. URL: <https://jupyter.org/> (дата обращения: 22.02.2025).
17. **Powers, D. M. W.**, "Evaluation: From Precision, Recall and F-measure to ROC, Informedness, Markedness and Correlation", Journal of Machine Learning Technologies, 2011. P. 37-58.
18. **Lowe, D. G.**, "Distinctive Image Features from Scale-Invariant Keypoints", International Journal of Computer Vision, 2004. P. 91-110.
19. **Everingham, M.**, et al., "The PASCAL Visual Object Classes Challenge", International Journal of Computer Vision, 2010. P. 303-350.
20. **Press, W. H.**, et al., "Numerical Recipes: The Art of Scientific Computing", Cambridge University Press, 2007. P. 180-190.
21. **Boyd, S., Vandenberghe, L.**, "Convex Optimization", Cambridge University Press, 2004. P. 145-160.

22. **Strang, G.**, "Introduction to Linear Algebra", Wellesley-Cambridge Press, 2016. P. 110-125.
23. **Cormen, T. H.**, Leiserson, C. E., Rivest, R. L., Stein, C., "Introduction to Algorithms", MIT Press, 2009. P. 180-200.
24. **Bishop, C. M.**, "Pattern Recognition and Machine Learning", Springer, 2006. P. 410-425.
25. **Pedregosa, F.**, et al., "Scikit-learn: Machine Learning in Python", Journal of Machine Learning Research, 2011. [Электронный ресурс]. URL: <https://scikit-learn.org/> (дата обращения: 24.03.2025).
26. **Hastie, T., Tibshirani, R., Friedman, J.**, "The Elements of Statistical Learning", Springer, 2009. P. 610-625.
27. **Zhou Z. H.**, "Ensemble Methods: Foundations and Algorithms", Chapman and Hall/CRC, 2012. P. 150-165.
28. **Kuhn, M., Johnson, K.**, "Feature Engineering and Selection: A Practical Approach for Predictive Models", Morgan Kaufmann, 2019. P. 220-235.
29. **Friedman, J. H.**, "Greedy Function Approximation: A Gradient Boosting Machine", Annals of Statistics, 2001. [Электронный ресурс]. URL: <https://projecteuclid.org/euclid.aos/1015951160> (дата обращения: 15.03.2025).

Приложение

GitHub [Электронный ресурс]. URL: <https://github.com/ciigann/Identification-of-patterns-of-behavior-of-laboratory-rats-by-sequence-of-video-frames.git>